

***Skip List – Insert Element – Program Analysis***  
***–Insertion (Insert At End)– Part 2***

***After First Insertion we have:***

*SkipList \* list*

*Address*      *list*

*ff33*      *ff08*

*Address*      *level*      *\* header*

*ff08*      2      *ff12*

*Address*      *key*      *\* forward*

*ff12*      -1      *ff80*

*Address*      *Values*

|                   |             |             |
|-------------------|-------------|-------------|
| <i>forward[0]</i> | <i>ff80</i> | <i>ff16</i> |
| <i>forward[1]</i> | <i>ff88</i> | <i>ff16</i> |
| <i>forward[2]</i> | <i>ff90</i> | <i>ff16</i> |
| <i>forward[3]</i> | <i>ff98</i> | <i>NULL</i> |

*Address*      *key*      *\* forward*

*ff16*      20      *0xb400*

*Address*      *Values*

|                   |               |             |
|-------------------|---------------|-------------|
| <i>forward[0]</i> | <i>0xb400</i> | <i>NULL</i> |
| <i>forward[1]</i> | <i>0xb408</i> | <i>NULL</i> |
| <i>forward[2]</i> | <i>0xb410</i> | <i>NULL</i> |

```
case 1:
    cout << "Enter key to insert: ";
    cin >> key;
    insertElement(list, key);
    break;
```

*Entering : Key = 30 .*

*insertElement(list: ff08 ,key: 30);*

*Now ,we have :*

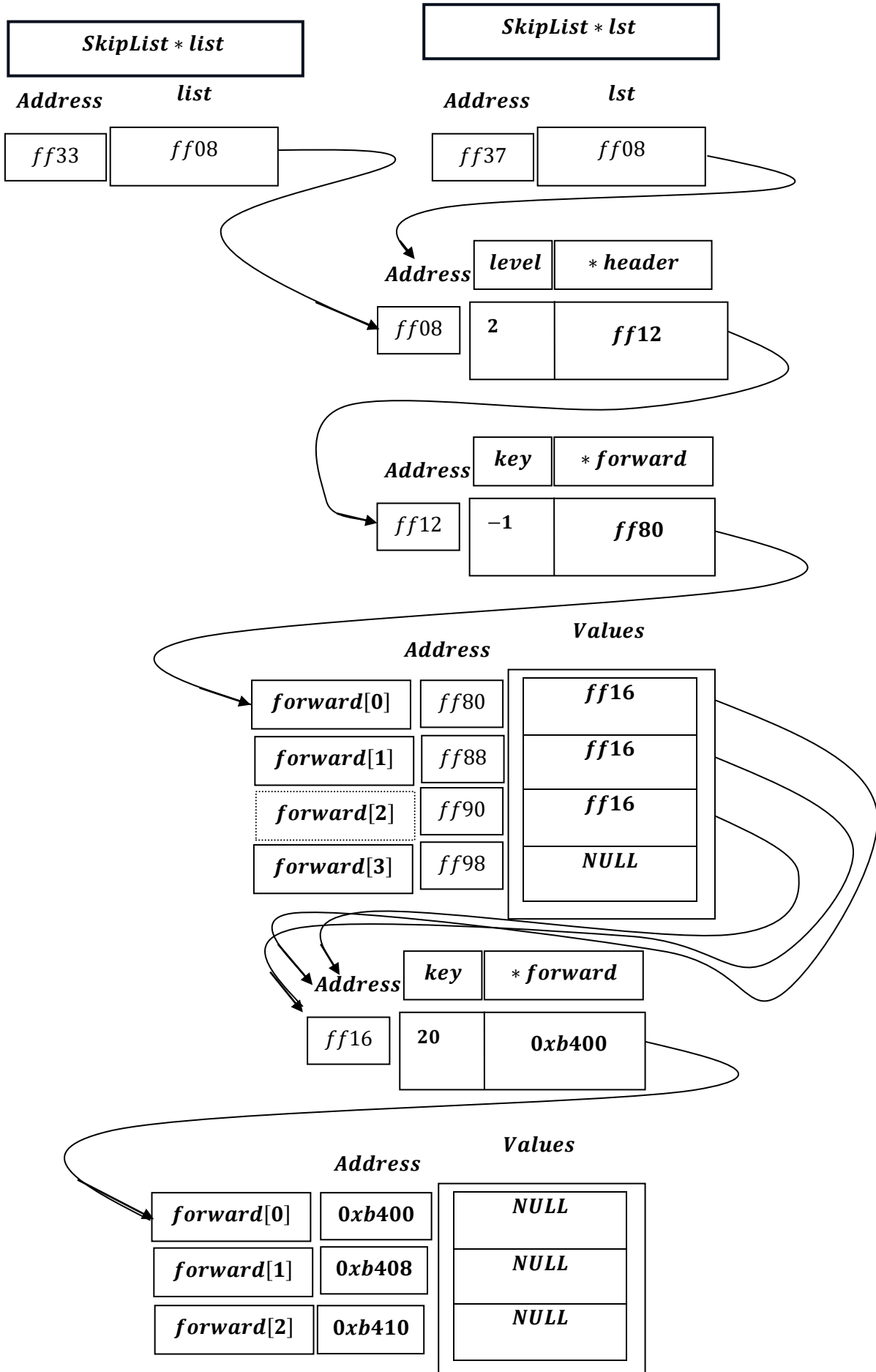
```
void insertElement(struct SkipList *lst, int key)
```

*list = struct SkipList \* lst = ff08 and int key: 30.*

*Now ,we have :*

```
void insertElement(struct SkipList *lst, int key)
```

*list = struct SkipList \* lst = ff08 and int key: 30.*



```
struct Node *current = lst->header;
```

*current = lst → header : ff12.*

```
struct Node *update[MAX_LEVEL + 1];
```

*update[3 + 1 = 4],*

*MAX\_LEVEL evaluates to 3. Consequently, the array size calculation is update[3 + 1 = 4].*

*This means the array is allocated to hold exactly 4 elements.*

*Before memset call, the update array may look like :*

| <i>Array Element</i> | <i>Hypothetical Stack Address</i> | <i>Value (Garbage Data)</i> | <i>Pointer Status</i>                                         |
|----------------------|-----------------------------------|-----------------------------|---------------------------------------------------------------|
| <i>update[0]</i>     | <i>0x7fff0000 to 0x7fff0007</i>   | <i>0x4a3f8921bce45099</i>   | <i>Wild Pointer<br/>(Points to a random, invalid address)</i> |
| <i>update[1]</i>     | <i>0x7fff0008 to 0x7fff000f</i>   | <i>0x0000000000000002</i>   | <i>Wild Pointer</i>                                           |
| <i>update[2]</i>     | <i>0x7fff0010 to 0x7fff0017</i>   | <i>0x7ffee1245a00881c</i>   | <i>Wild Pointer</i>                                           |
| <i>update[3]</i>     | <i>0x7fff0018 to 0x7fff001f</i>   | <i>0x0000000000000000</i>   | <i>Accidentally NULL<br/>(Just by chance)</i>                 |

*More simpler way:*

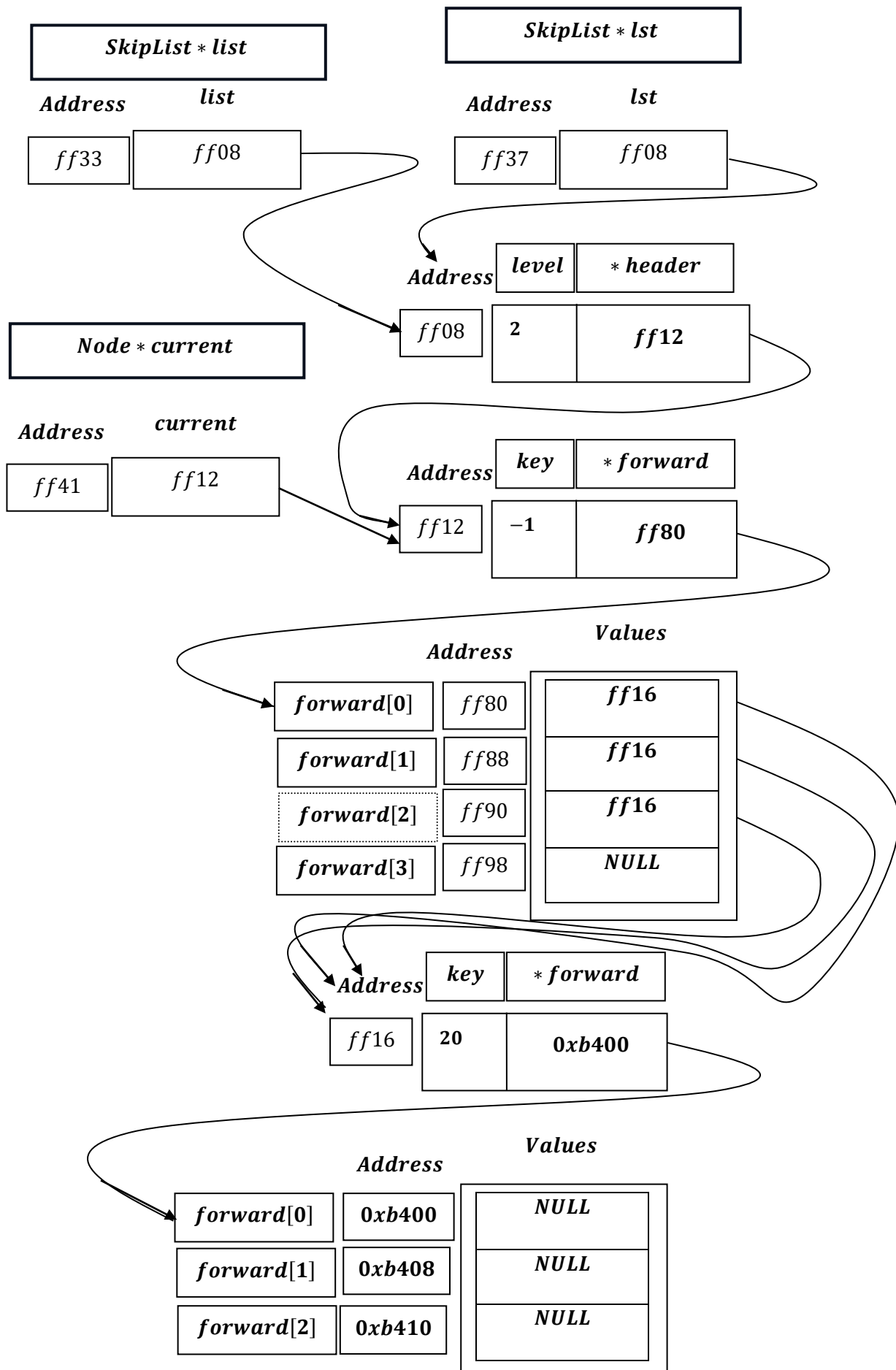
|           | Address    | Values             |           |
|-----------|------------|--------------------|-----------|
| update[0] | 0x7fff0000 | 0x4a3f8921bce45099 | [8 bytes] |
| update[1] | 0x7fff0008 | 0x0000000000000002 | [8 bytes] |
| update[2] | 0x7fff0010 | 0x7ffee1245a00881c | [8 bytes] |
| update[3] | 0x7fff0018 | 0x0000000000000000 | [8 bytes] |

```
memset(update, 0, sizeof(struct Node *) * (MAX_LEVEL + 1));
```

*``memset`` stands for Memory Set. It takes a block of memory and rapidly fills it with a specific value. Here `0` or NULL.*

|           | Address    | Values |
|-----------|------------|--------|
| update[0] | 0x7fff0000 | NULL   |
| update[1] | 0x7fff0008 | NULL   |
| update[2] | 0x7fff0010 | NULL   |
| update[3] | 0x7fff0018 | NULL   |

*Here we kept addresses same like: 0x7fff0000 , just for an example , but it may not be same , in reality.*



|                  | <i>Address</i> | <i>Values</i> |
|------------------|----------------|---------------|
| <i>update[0]</i> | 0x7fff0000     | <i>NULL</i>   |
| <i>update[1]</i> | 0x7fff0008     | <i>NULL</i>   |
| <i>update[2]</i> | 0x7fff0010     | <i>NULL</i>   |
| <i>update[3]</i> | 0x7fff0018     | <i>NULL</i>   |

```

for (int i = lst->level; i >= 0; i--) {
    while (current->forward[i] != NULL && current->forward[i]->key <
key) {
        current = current->forward[i];
    }
    update[i] = current;
}

```

*For :  $i = lst \rightarrow level = 2 ; i \geq 0$   $\left[ \begin{array}{l} \text{Condition} \\ \text{is True} \end{array} \right]$*

*while:  $current \rightarrow forward[2] = ff16 \neq NULL$   $\left[ \begin{array}{l} \text{Condition} \\ \text{is True} \end{array} \right]$  And*

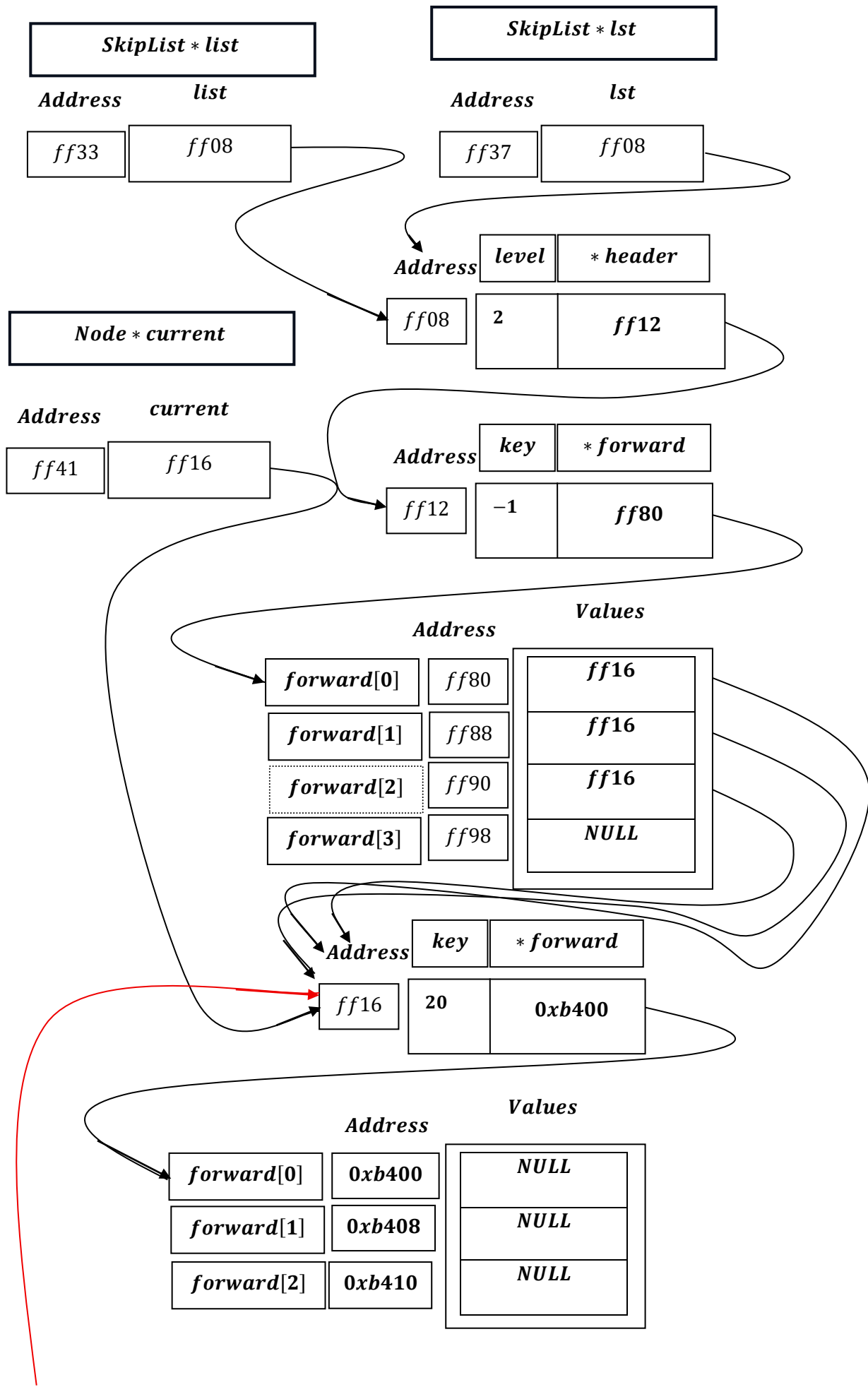
*$current \rightarrow forward[2] \rightarrow key: 20 < key: 30$   $\left[ \begin{array}{l} \text{Condition} \\ \text{is True} \end{array} \right]$*

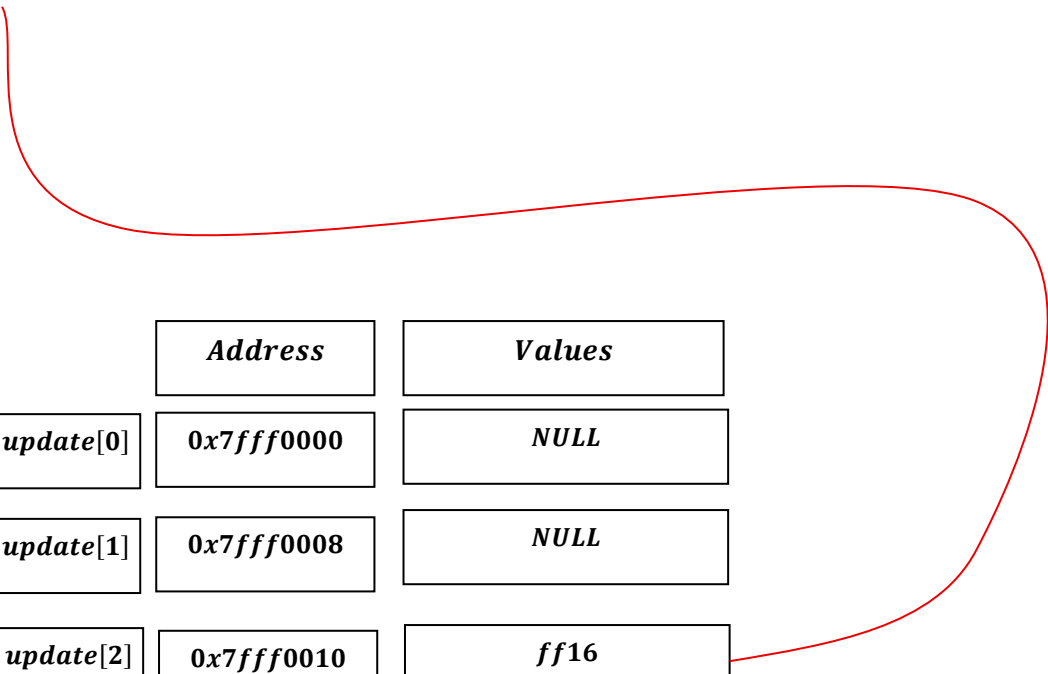
*$current = current \rightarrow forward[2] = ff16$*

*while:  $current \rightarrow forward[2] = NULL \neq NULL$   $\left[ \begin{array}{l} \text{Condition} \\ \text{is False, while} \\ \text{loop exits} \end{array} \right]$*

*$update[2] = current = ff16.$*

*$i = i - 1 = 2 - 1 = 1.$*





|                  | <i>Address</i>    | <i>Values</i> |
|------------------|-------------------|---------------|
| <i>update[0]</i> | <i>0x7fff0000</i> | <i>NULL</i>   |
| <i>update[1]</i> | <i>0x7fff0008</i> | <i>NULL</i>   |
| <i>update[2]</i> | <i>0x7fff0010</i> | <i>ff16</i>   |
| <i>update[3]</i> | <i>0x7fff0018</i> | <i>NULL</i>   |

*For :  $i = 1 ; i \geq 0$   $\left[ \begin{array}{l} \text{Condition} \\ \text{is True} \end{array} \right]$*

*while:  $current \rightarrow forward[1] = NULL \neq NULL$   $\left[ \begin{array}{l} \text{Condition} \\ \text{is False} \end{array} \right]$*

*[hence, while loop exits]*

*$update[1] = current = ff16.$*

*$i = i - 1 = 1 - 1 = 0.$*

*SkipList \* list*

Address list

|      |      |
|------|------|
| ff33 | ff08 |
|------|------|

*SkipList \* lst*

Address lst

|      |      |
|------|------|
| ff37 | ff08 |
|------|------|

*Node \* current*

Address current

|      |      |
|------|------|
| ff41 | ff16 |
|------|------|

Address level \* header

|      |   |      |
|------|---|------|
| ff08 | 2 | ff12 |
|------|---|------|

Address key \* forward

|      |    |      |
|------|----|------|
| ff12 | -1 | ff80 |
|------|----|------|

Address Values

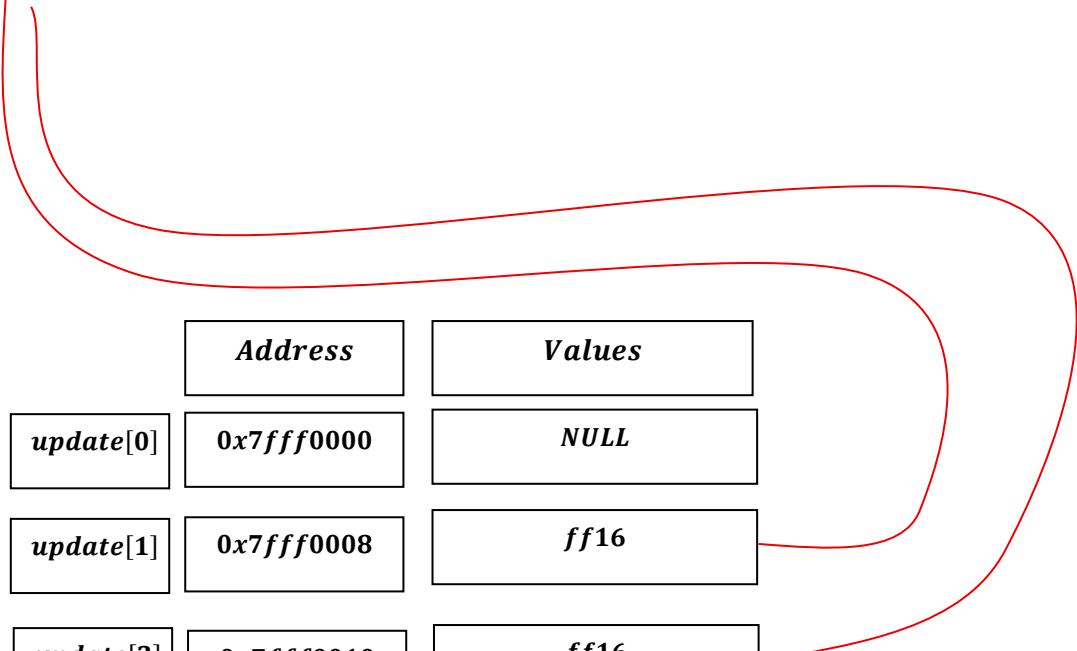
| Address    | Values |      |
|------------|--------|------|
| forward[0] | ff80   | ff16 |
| forward[1] | ff88   | ff16 |
| forward[2] | ff90   | ff16 |
| forward[3] | ff98   | NULL |

Address key \* forward

|      |    |        |
|------|----|--------|
| ff16 | 20 | 0xb400 |
|------|----|--------|

Address Values

| Address    | Values |      |
|------------|--------|------|
| forward[0] | 0xb400 | NULL |
| forward[1] | 0xb408 | NULL |
| forward[2] | 0xb410 | NULL |



|                  | <i>Address</i>    | <i>Values</i> |
|------------------|-------------------|---------------|
| <i>update[0]</i> | <i>0x7fff0000</i> | <i>NULL</i>   |
| <i>update[1]</i> | <i>0x7fff0008</i> | <i>ff16</i>   |
| <i>update[2]</i> | <i>0x7fff0010</i> | <i>ff16</i>   |
| <i>update[3]</i> | <i>0x7fff0018</i> | <i>NULL</i>   |

*For :  $i = 0 ; i \geq 0$   $\left[ \begin{array}{l} \text{Condition} \\ \text{is True} \end{array} \right]$*

*while:  $current \rightarrow forward[0] = NULL \neq NULL$   $\left[ \begin{array}{l} \text{Condition} \\ \text{is False} \end{array} \right]$*

*[hence, while loop exits]*

*$update[0] = current = ff16.$*

*$i = i - 1 = 0 - 1 = -1.$*

**SkipList \* list**

Address list

|      |      |
|------|------|
| ff33 | ff08 |
|------|------|

**SkipList \* lst**

Address lst

|      |      |
|------|------|
| ff37 | ff08 |
|------|------|

**Node \* current**

Address current

|      |      |
|------|------|
| ff41 | ff16 |
|------|------|

Address level \* header

|      |   |      |
|------|---|------|
| ff08 | 2 | ff12 |
|------|---|------|

Address key \* forward

|      |    |      |
|------|----|------|
| ff12 | -1 | ff80 |
|------|----|------|

Address Values

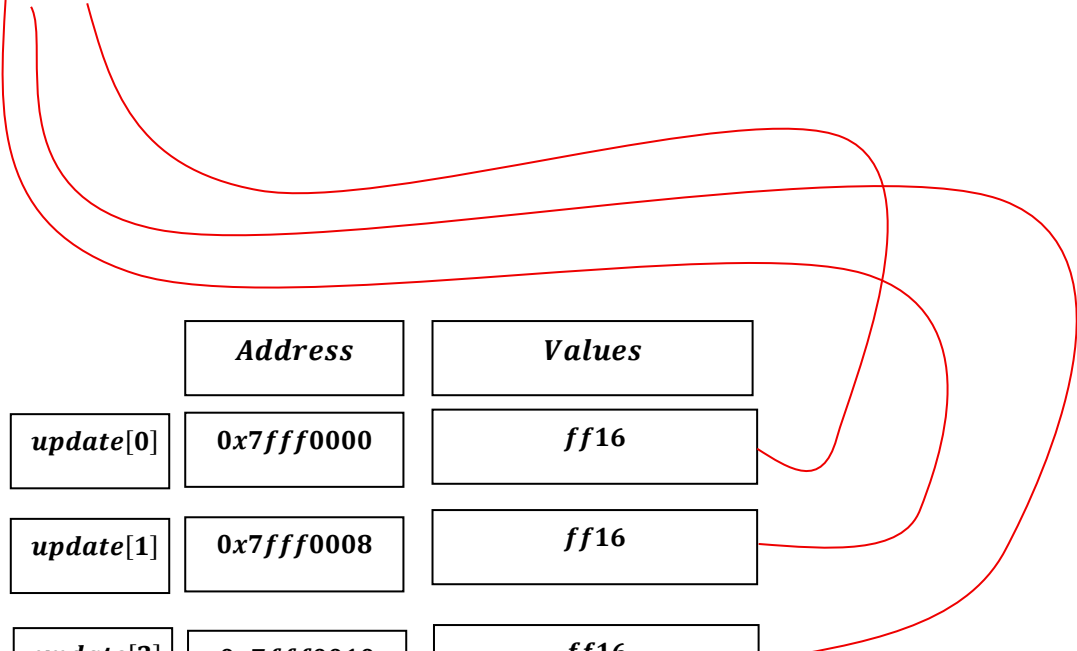
| Address    | Values |      |
|------------|--------|------|
| forward[0] | ff80   | ff16 |
| forward[1] | ff88   | ff16 |
| forward[2] | ff90   | ff16 |
| forward[3] | ff98   | NULL |

Address key \* forward

|      |    |        |
|------|----|--------|
| ff16 | 20 | 0xb400 |
|------|----|--------|

Address Values

| Address    | Values |      |
|------------|--------|------|
| forward[0] | 0xb400 | NULL |
| forward[1] | 0xb408 | NULL |
| forward[2] | 0xb410 | NULL |



|                  | <i>Address</i> | <i>Values</i> |
|------------------|----------------|---------------|
| <i>update[0]</i> | 0x7fff0000     | ff16          |
| <i>update[1]</i> | 0x7fff0008     | ff16          |
| <i>update[2]</i> | 0x7fff0010     | ff16          |
| <i>update[3]</i> | 0x7fff0018     | NULL          |

*For :  $i = -1 ; i \geq 0$   $\left[ \begin{array}{l} \text{Condition} \\ \text{is False} \end{array} \right]$ , condition is false and loop exits.*

```
current = current->forward[0];
```

*current = current  $\rightarrow$  forward[0] = NULL.*

```
if (current == NULL || current->key != key) {...}
```

*current = NULL (True)*

```
int rlevel = randomLevel();
```

*rlevel = randomlevel( ) = 1 (say)*

```
if (rlevel > lst->level) {
    for (int i = lst->level + 1; i <= rlevel; i++) {
        update[i] = lst->header;
    }
    lst->level = rlevel;
}
```

*if  $rlevel = 1 > lst \rightarrow level = 2$  is false  $\left[ \begin{array}{c} \text{Condition is} \\ \text{false} \end{array} \right]$ , if block exits.*

*SkipList \* list*

*Address list*

|             |             |
|-------------|-------------|
| <i>ff33</i> | <i>ff08</i> |
|-------------|-------------|

*SkipList \* lst*

*Address lst*

|             |             |
|-------------|-------------|
| <i>ff37</i> | <i>ff08</i> |
|-------------|-------------|

*Node \* current*

*Address current*

|             |             |
|-------------|-------------|
| <i>ff41</i> | <b>NULL</b> |
|-------------|-------------|

*Address level \* header*

|             |   |             |
|-------------|---|-------------|
| <i>ff08</i> | 2 | <i>ff12</i> |
|-------------|---|-------------|

*Address key \* forward*

|             |    |             |
|-------------|----|-------------|
| <i>ff12</i> | -1 | <i>ff80</i> |
|-------------|----|-------------|

*Address Values*

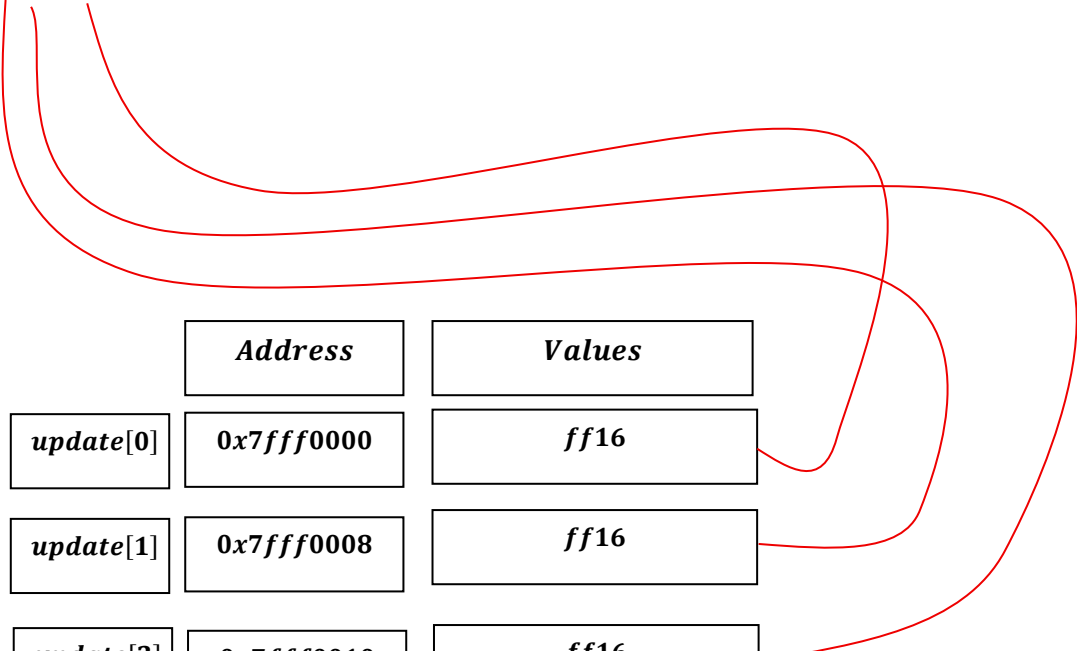
| <i>forward</i>    | <i>Address</i> | <i>Values</i> |
|-------------------|----------------|---------------|
| <i>forward[0]</i> | <i>ff80</i>    | <i>ff16</i>   |
| <i>forward[1]</i> | <i>ff88</i>    | <i>ff16</i>   |
| <i>forward[2]</i> | <i>ff90</i>    | <i>ff16</i>   |
| <i>forward[3]</i> | <i>ff98</i>    | <b>NULL</b>   |

*Address key \* forward*

|             |    |               |
|-------------|----|---------------|
| <i>ff16</i> | 20 | <b>0xb400</b> |
|-------------|----|---------------|

*Address Values*

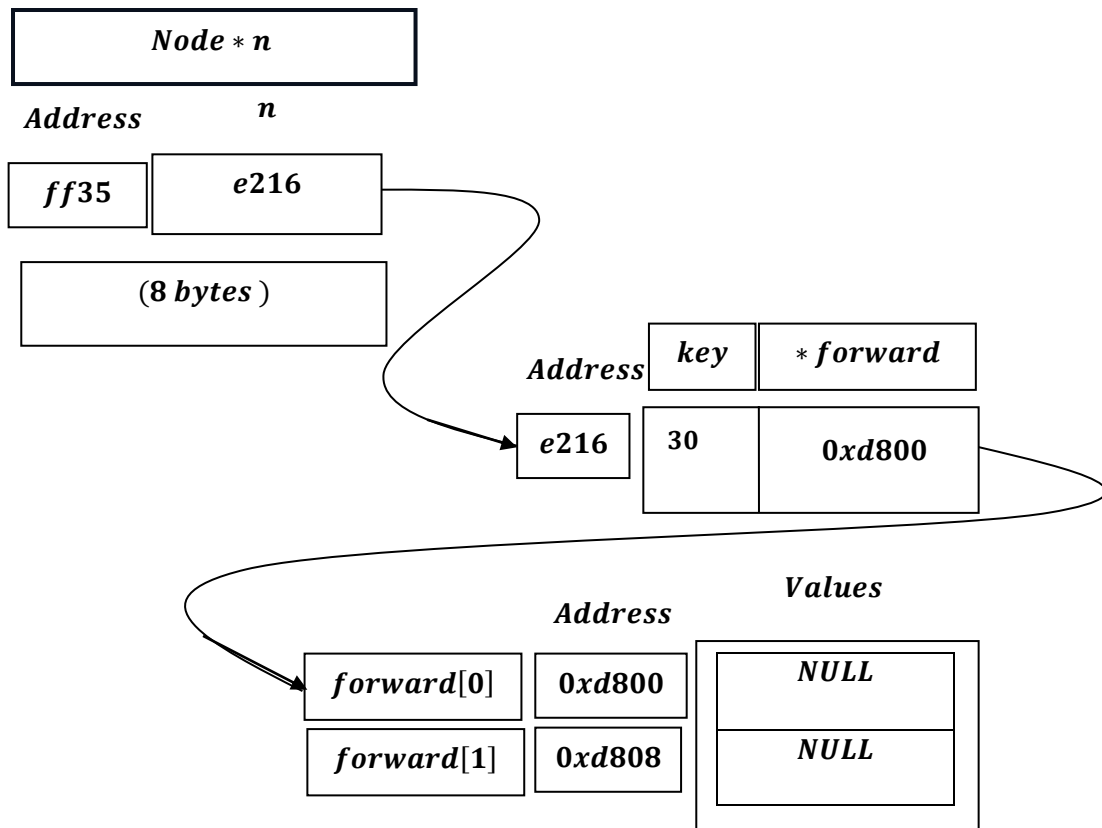
| <i>forward</i>    | <i>Address</i> | <i>Values</i> |
|-------------------|----------------|---------------|
| <i>forward[0]</i> | <b>0xb400</b>  | <b>NULL</b>   |
| <i>forward[1]</i> | <b>0xb408</b>  | <b>NULL</b>   |
| <i>forward[2]</i> | <b>0xb410</b>  | <b>NULL</b>   |



|                  | <i>Address</i>    | <i>Values</i> |
|------------------|-------------------|---------------|
| <i>update[0]</i> | <i>0x7fff0000</i> | <i>ff16</i>   |
| <i>update[1]</i> | <i>0x7fff0008</i> | <i>ff16</i>   |
| <i>update[2]</i> | <i>0x7fff0010</i> | <i>ff16</i>   |
| <i>update[3]</i> | <i>0x7fff0018</i> | <i>NULL</i>   |

```
struct Node *n = createNode(key, rlevel){...};
```

*Let rlevel = 1 and key = 30.*



```
return n;
```

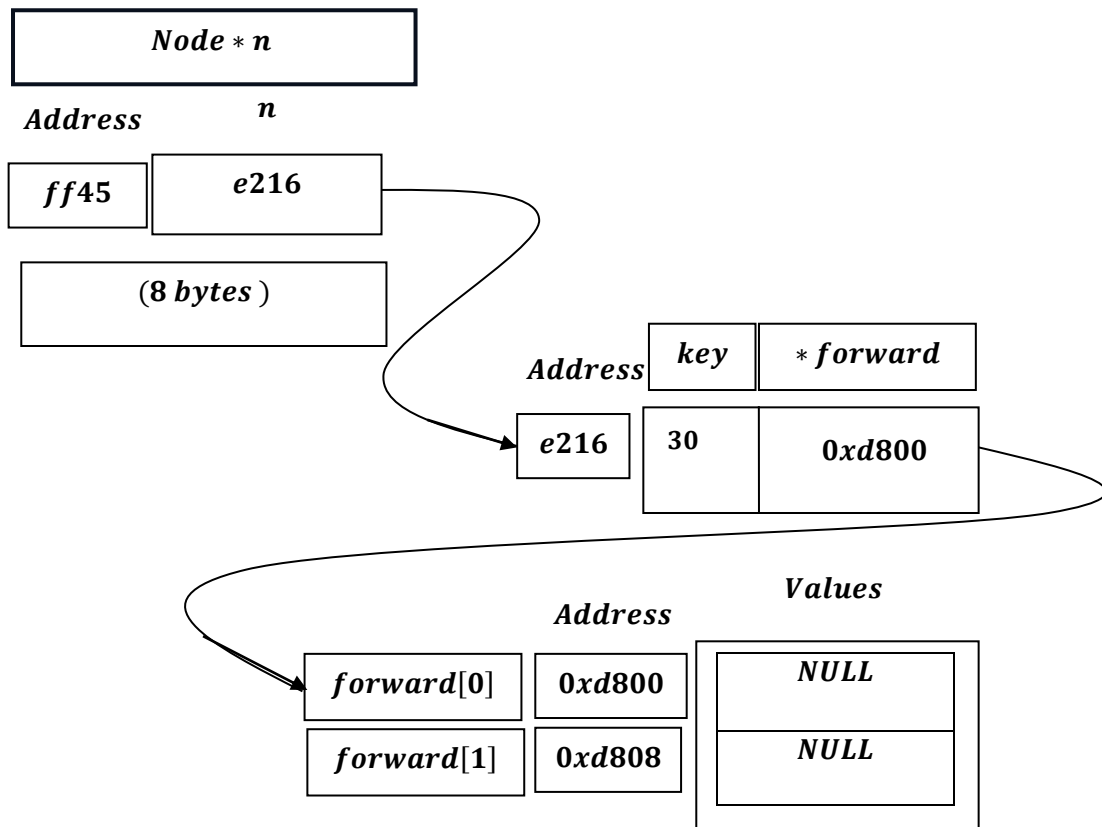
i. e. Return  $n = e216$ .

### Actual structure

| Address at which value is stored | Value           | offset used                                                    |
|----------------------------------|-----------------|----------------------------------------------------------------|
| e216                             | key = 30        | key[e216 – e219]<br>4 bytes for int                            |
|                                  |                 | padding → [e21a – e21d]<br>4 bytes for padding                 |
| e21e                             | Address: 0xd800 | * forward → [e21e – e225]<br>8 bytes for pointer variable      |
| 0xd800                           | NULL            | forward[0] → [0xd800 – 0xd807]<br>8 bytes for pointer variable |
| 0xd808                           | NULL            | forward[1] → [0xd808 – 0xd80f]<br>8 bytes for pointer variable |

***struct Node \* n = createNode(key = 30, rlevel = 1)***

***struct Node \* n = e216***



```
for (int i = 0; i <= rlevel; i++) {
    n->forward[i] = update[i]->forward[i];
    update[i]->forward[i] = n;
}
```

***int i = 0; i ≤ rlevel = 1***

***n → forward[0] = update[0] → forward[0] = NULL***

***update[0] → forward[0] = n = e216***

***i = i + 1 = 0 + 1 = 1***

***int i = 1; i ≤ rlevel = 1***

***n → forward[1] = update[1] → forward[1] = NULL***

***update[1] → forward[1] = n = e216***

***i = i + 1 = 1 + 1 = 2***

***int i = 2; i ≤ rlevel = 1* [ *Condition*  
*is False* ]**

*SkipList \* list*

*Address list*

|             |             |
|-------------|-------------|
| <i>ff33</i> | <i>ff08</i> |
|-------------|-------------|

*SkipList \* lst*

*Address lst*

|             |             |
|-------------|-------------|
| <i>ff37</i> | <i>ff08</i> |
|-------------|-------------|

*Node \* current*

*Address current*

|             |             |
|-------------|-------------|
| <i>ff41</i> | <i>NULL</i> |
|-------------|-------------|

*Address level \* header*

|             |          |             |
|-------------|----------|-------------|
| <i>ff08</i> | <i>2</i> | <i>ff12</i> |
|-------------|----------|-------------|

*Address key \* forward*

|             |           |             |
|-------------|-----------|-------------|
| <i>ff12</i> | <i>-1</i> | <i>ff80</i> |
|-------------|-----------|-------------|

*Address Values*

| <i>forward</i>    | <i>Address</i> | <i>Values</i> |
|-------------------|----------------|---------------|
| <i>forward[0]</i> | <i>ff80</i>    | <i>ff16</i>   |
| <i>forward[1]</i> | <i>ff88</i>    | <i>ff16</i>   |
| <i>forward[2]</i> | <i>ff90</i>    | <i>ff16</i>   |
| <i>forward[3]</i> | <i>ff98</i>    | <i>NULL</i>   |

*Address key \* forward*

|             |           |               |
|-------------|-----------|---------------|
| <i>ff16</i> | <i>20</i> | <i>0xb400</i> |
|-------------|-----------|---------------|

*Address Values*

| <i>forward</i>    | <i>Address</i> | <i>Values</i> |
|-------------------|----------------|---------------|
| <i>forward[0]</i> | <i>0xb400</i>  | <i>e216</i>   |
| <i>forward[1]</i> | <i>0xb408</i>  | <i>e216</i>   |
| <i>forward[2]</i> | <i>0xb410</i>  | <i>NULL</i>   |

|                  | <i>Address</i> | <i>Values</i> |
|------------------|----------------|---------------|
| <i>update[0]</i> | 0x7fff0000     | <i>ff16</i>   |
| <i>update[1]</i> | 0x7fff0008     | <i>ff16</i>   |
| <i>update[2]</i> | 0x7fff0010     | <i>ff16</i>   |
| <i>update[3]</i> | 0x7fff0018     | <i>NULL</i>   |

*Node \* n*

*Address*      *n*

*ff45*      *e216*

(8 bytes )

| <i>Address</i> | <i>key</i> | <i>* forward</i> |
|----------------|------------|------------------|
| <i>e216</i>    | 30         | 0xd800           |

|                   | <i>Address</i> | <i>Values</i> |
|-------------------|----------------|---------------|
| <i>forward[0]</i> | 0xd800         | <i>NULL</i>   |
| <i>forward[1]</i> | 0xd808         | <i>NULL</i>   |

```
cout << "Successfully inserted key " << key << endl;
```

*Successfully inserted key : 30 and function ends .*

*As , the function ends :*

*SkipList \* lst*

| Address | lst  |
|---------|------|
| ff37    | ff08 |

*Node \* current*

| Address | current |
|---------|---------|
| ff41    | NULL    |

|           | Address    | Values |
|-----------|------------|--------|
| update[0] | 0x7fff0000 | ff16   |
| update[1] | 0x7fff0008 | ff16   |
| update[2] | 0x7fff0010 | ff16   |
| update[3] | 0x7fff0018 | NULL   |

*Node \* n*

| Address | n    |
|---------|------|
| ff45    | e216 |

*Gets  
Destroyed*

And we are left with :

